ADR-004 — Durabilidade e escalabilidade (20x o volume)

Oct 19, 2026 · @Mahalanobis

Contexto

Na Black Friday a ShopVibe precisa suportar 20 vezes o volume normal sem perder nenhum pedido, mesmo que um componente caia. Perder pedidos significaria vendas e dinheiro desaparecidos, e perder eventos de fraude significaria novos chargebacks.

O volume é modesto para um sistema de streaming. Supondo cerca de 5 eventos por pedido (o pedido, a atualização do total e os itens), a conta é:

Cenário	Pedidos/dia	Pedidos/s (média)	Eventos/s (média)
Dia normal	15.000	0,17	0,9
Black Friday (20x)	300.000	3,5	17

Mesmo que o pico real seja 10 vezes a média da Black Friday, ficamos perto de 170 eventos por segundo, bem abaixo do que um broker Kafka comum sustenta. O desafio aqui é a durabilidade e a capacidade de crescer, não a vazão bruta.

Decisão

Garantimos entrega at-least-once de ponta a ponta com quatro mecanismos, e escalamos horizontalmente por partições, em vez de usar máquinas maiores.

1. **Origem:** o slot de replicação do PostgreSQL guarda o log enquanto o Debezium está fora do ar, e o conector retoma do último offset confirmado.
2. **Kafka:** em produção, fator de replicação 3, `min.insync.replicas=2` e produtores com `acks=all`. O laboratório usa um único broker com fator 1, então não demonstra essa proteção.
3. **Bronze:** o writer desliga o auto-commit e só confirma o offset depois de gravar o arquivo no S3. O nome do arquivo é determinístico, então reprocessar um lote não cria cópias novas.
4. **Escala:** cada tópico tem 3 partições no laboratório, o que permite até 3 instâncias do writer no mesmo consumer group. Para a Black Friday, aumentamos as partições (por exemplo, 12) e as instâncias, e acrescentamos workers ao Kafka Connect e brokers ao cluster.

Alternativas consideradas

Preferimos aceitar duplicatas e removê-las depois a correr o risco de perder um pedido.

Alternativa	Por que não foi escolhida
Exactly-once (transações do Kafka e destinos transacionais)	Elimina duplicatas, mas aumenta muito a complexidade; a deduplicação por tópico, partição e offset na silver é barata

Alternativa	Por que não foi escolhida
Auto-commit do consumidor	Confirma o offset antes de gravar; se o writer cair nesse intervalo, o evento é perdido
Escala vertical (um broker maior)	Tem teto e continua sendo ponto único de falha, para um volume que cabe em hardware modesto
Manter fator de replicação 1 em produção	Perder o disco do broker significaria perder pedidos
Um arquivo por evento na bronze	Multiplica arquivos minúsculos no S3 e piora a leitura

Consequências

O desenho tolera a queda de qualquer componente isolado e cresce com folga para 20 vezes o volume, mas o laboratório só demonstra a captura e a gravação, não a tolerância a falhas.

Positivas

- Nenhum pedido se perde se o writer, o Connect, o broker ou o S3 caírem um de cada vez, desde que a configuração de produção esteja aplicada.
- Cada etapa escala de forma independente: partições e writers para ler, workers para capturar, brokers para armazenar.
- A carga estimada da Black Friday (cerca de 17 eventos/s em média) deixa grande margem de capacidade.

Negativas

- A entrega é at-least-once: haverá eventos duplicados, e as camadas seguintes precisam deduplicar por tópico, partição e offset.
- Replicação 3x triplica o disco do Kafka e adiciona latência de confirmação.
- O conector PostgreSQL do Debezium usa uma única tarefa por conector, então a captura não se divide em paralelo. A margem de volume compensa, mas é um teto a monitorar.
- Aumentar as partições muda o mapeamento de chave para partição, e a ordem dos eventos por registro pode se embaralhar durante a transição.
- O laboratório roda com um único nó de cada componente e fator de replicação 1: a durabilidade aqui é argumentada, não demonstrada.
- O slot de replicação pode encher o disco do PostgreSQL se o Debezium ficar parado por muito tempo (ver ADR-001).